

Algorithms and data structures.....	1
Introduction.....	1
Set up and file reading.....	1
Task 1 - BubbleSort.....	2
a) Implementation.....	2
Difference between optimized and non-optimized Bubble Sort.....	2
b) Time complexity.....	3
Non-optimized Bubble Sort:.....	3
Optimized Bubble Sort:.....	4
Does shuffling change the time complexity?.....	4
Task 2 - InsertionSort.....	5
a) Implementation.....	5
b) Time complexity.....	6
Does shuffling change the time complexity?.....	6
Task 3 - MergeSort.....	7
a) Implementation.....	7
b) Number of merge operations - does shuffling change it?.....	8
Task 4 - QuickSort.....	9
a) Implementation.....	9
a) First element:.....	10
b) Last element:.....	10
c) Random element:.....	10
d) Median of three:.....	10
b) Comparisons for each pivot strategy.....	12
a) The number of comparisons.....	12
b) Which pivot selection strategy works best.....	12
Conclusion.....	13
Reflections.....	13
References.....	14

Algorithms and data structures

Introduction

For this exam in PG4200 Data Structures and Algorithms, we have based our work on the Wine Quality Dataset, consisting of physicochemical measurements of red and white Portuguese "Vinho Verde" wine samples (Cortez et al., 2009). The dataset contains 1,599 red wine samples and 4,898 white wine samples, each with 11 input variables including alcohol content, acidity, and pH. From these combined datasets, we extracted the unique alcohol content values to use as input for our sorting algorithms. We will analyse the implementation of the different algorithms with this data set, as well as the time complexity and how they compare to one another.

Set up and file reading

The code solution is structured across eight classes, each responsible for a distinct part of the implementation. All the sorting classes each contain their own main method, while the ReadFromFile class serves as a shared utility that is called from within each sorting class to handle data import.

The full class structure is as follows: ReadFromFile, Task1_BubbleSort_NonOptimised, Task1_BubbleSort_Optimised, Task2_InsertionSort, Task3_MergeSort, Task4_QuickSort, Wine and WineType. In addition, a separate data package contains the two input files that are processed by the ReadFromFile class.

The Wine record represents a single wine entry as a data object, storing the relevant attributes parsed from the CSV files. The WineType enum class is used to distinguish between red and white wine entries, allowing the combined dataset to be handled in a type-safe manner. The ReadFromFile class reads both CSV files, parses each row into Wine objects using ArrayLists, and then each class extracts the alcohol content using HashSets which is used to implement the various sorting algorithms.

Since the CSV files use Windows-style line endings, the last field on each line (values[11], representing quality) may contain a trailing `\r` character that is invisible but would cause a `NumberFormatException` when parsing the value as an integer. To prevent this, `.trim()` is added to `scanner.nextLine()`.

```
while (scanner.hasNextLine()) {
    String line = scanner.nextLine().trim();

    String[] values = line.split(",");

    Wine wine = new Wine(
        Double.parseDouble(values[0]),
        Double.parseDouble(values[1]),
        Double.parseDouble(values[2]),
        Double.parseDouble(values[3]),
        Double.parseDouble(values[4]),
        Double.parseDouble(values[5]),
        Double.parseDouble(values[6]),
        Double.parseDouble(values[7]),
        Double.parseDouble(values[8]),
        Double.parseDouble(values[9]),
        Double.parseDouble(values[10]),
        Integer.parseInt(values[11]),
        type
    );

    wines.add(wine);
}
```

Task 1 - BubbleSort

a) Implementation

Bubble Sort is a simple sorting algorithm that works by repeatedly comparing and swapping adjacent elements if they are in the wrong order (GeeksforGeeks, n.d.-a). In other words, it sorts a sequence of S and n comparable elements (Goodrich & Tamassia, 2006, p. 370). The algorithm repeatedly swaps adjacent elements until the list is fully sorted (Marculescu, n.d.). After the first pass, the largest element has bubbled up to the last position (Marculescu, n.d.). After the second pass, the second largest is in its correct position, and so on until the entire list is sorted (GeeksforGeeks, n.d.-a). This sorting algorithm has a high worst-case time complexity and is not efficient in large data sets (GeeksforGeeks, n.d.-a). Out of the four we have implemented in this exam, Bubble Sort is the easiest to implement, but least efficient, with a worst-case time complexity of $O(n^2)$ (GeeksforGeeks, n.d.-a).

Difference between optimized and non-optimized Bubble Sort

Two variants are implemented in this solution: a non-optimized and an optimized version, both sorting the unique alcohol content values from the combined red and white wine dataset in ascending order.

The **non-optimized** version uses two nested loops (Marculescu, n.d.). The outer loop runs exactly $n-1$ times, and the inner loop compares and swaps adjacent elements as needed (Marculescu, n.d.). Crucially, the outer loop has no mechanism to stop early. It always completes all $n-1$ passes, even if the list becomes sorted before the final pass (Marculescu, n.d.). This means the algorithm always performs the same amount of work regardless of the initial order of the data (GeeksforGeeks, n.d.-a).

```
public static int[] bubbleSort(ArrayList<Double> list) {
    int iteration = 0;
    int swap = 0;

    // Outer loop - will run exactly n-1 times
    for (int i = 0; i < list.size() - 1; i++) {
        iteration++;
        // Inner loop compares and swaps adjacent elements
        for (int j = 0; j < list.size() - 1 - i; j++) {
            if (list.get(j) > list.get(j + 1)) {
                double temp = list.get(j);
                list.set(j, list.get(j + 1));
                list.set(j + 1, temp);
                swap++;
            }
        }
    }

    return new int[]{
        iteration,
        swap
    };
}
```

The **optimized** version adds a swapped flag inside the outer loop (Interviewer live, 2023). If a full pass completes without any swaps, the flag remains false and the algorithm breaks out of the loop early (Interviewer live, 2023). This is the key optimization: the algorithm needs one whole pass without any swap to confirm that the list is sorted (Interviewer live, 2023). On a nearly sorted or already sorted list the optimized version can therefore finish significantly faster, while on a completely unsorted list both variants perform similarly (GeeksforGeeks, n.d.-a, Interviewer live, 2023).

```
public static int[] bubbleSort(ArrayList<Double> list) {
    int iteration = 0;
    int swap = 0;

    // Outer loop - will run max n-1 times
    for (int i = 0; i < list.size() - 1; i++) {
        iteration++;
        boolean swapped = false;

        // Inner loop - compares and swaps adjacent elements
        for (int j = 0; j < list.size() - 1 - i; j++) {
            if (list.get(j) > list.get(j + 1)) {
                double temp = list.get(j);
                list.set(j, list.get(j + 1));
                list.set(j + 1, temp);
                swap++;
                swapped = true;
            }
        }

        // If no elements were swapped this pass, the list is sorted
        if (!swapped) break;
    }

    return new int[]{
        iteration,
        swap
    };
}
```

b) Time complexity

The time complexity of Bubble Sort is determined by the two nested loops. Since we have two nested loops of size n , the outer loop runs $n-1$ times and for each iteration of the outer loop, the inner loop runs up to $n-1-i$ times (Codecademy, n.d.). This gives the following complexities:

Non-optimized Bubble Sort:

- Best case: $O(n^2)$ - even if the list is already sorted, all $n-1$ passes always run (GeeksforGeeks, n.d.,-e)
- Average case: $O(n^2)$ (GeeksforGeeks, n.d.,-e)
- Worst case: $O(n^2)$ (GeeksforGeeks, n.d.,-e)

```

=== Bubble Sort (Non-Optimised) ===

[Natural order]
Before: [8.8, 9.5, 10.1, 9.9, 9.6, 11.2]
After:  [8.0, 8.4, 8.5, 8.6, 8.7, 8.8]
Passes: 110
Swaps:  2697

[Shuffled]
Before: [12.25, 9.6, 12.3, 8.9, 10.1, 11.2]
After:  [8.0, 8.4, 8.5, 8.6, 8.7, 8.8]
Passes: 110
Swaps:  3128

```

Optimized Bubble Sort:

- Best case: $O(n)$ - if the list is already sorted, only one pass is needed before the swapped flag triggers an early exit (Alake, 2025)
- Average case: $O(n^2)$ (Alake, 2025)
- Worst case: $O(n^2)$ - a reverse-sorted list produces no early exits (Alake, 2025)

```

[Natural order]
Before: [8.8, 9.5, 10.1, 9.9, 9.6, 11.2]
After:  [8.0, 8.4, 8.5, 8.6, 8.7, 8.8]
Passes: 102
Swaps:  2697

[Shuffled]
Before: [9.8, 10.1, 12.3, 8.9, 10.1, 11.2]
After:  [8.0, 8.4, 8.5, 8.6, 8.7, 8.8]
Passes: 94
Swaps:  3162

```

The space complexity for both variants is $O(1)$, since the memory required stays constant regardless of the size input (GeeksforGeeks, n.d.,-e, Alake, 2025).

Does shuffling change the time complexity?

For the **non-optimized** version, shuffling does not change the time complexity. The outer loop always runs $n-1$ times regardless of input order, so the complexity stays $O(n^2)$ in all cases (Marculescu, n.d.).

For the **optimized** version, shuffling can increase the actual number of passes and swaps. A randomly shuffled list is unlikely to trigger early exits in the outer loop, meaning the algorithm runs closer to its worst case. The time complexity remains $O(n^2)$, but the constant factor increases. This is demonstrated in the program output, which runs both algorithms on the original list and a shuffled version and prints the number of passes and swaps for direct comparison.

Task 2 - InsertionSort

a) Implementation

Insertion Sort works similarly to how you would sort a hand of playing cards. Imagine you are holding a pile of unsorted cards. You pick up the first card since it's the only one, it's already "sorted." Then you pick the next card and compare it to the one you are already holding. If it's smaller, you place it accordingly to where it should go in the left already sorted pile.

For each new card you pick up, you scan through the sorted cards from right to left, shifting each larger card one position until you find the correct spot. Then you insert the card into that gap (Marculescu, n.d.).

In code, the key variable represents the card you just picked up. The loop performs the scanning process by iterating through the sorted portion and shifting elements that are larger than the key one step to the right. When the loop stops, we have found the correct position, and key is inserted there (Goodrich & Tamassia, 2006, p. 150).

```
public static int[] insertionSort(ArrayList<Double> list) {  
    int passes = 0;  
    int shifts = 0;  
    int comparisons = 0;  
    int n = list.size();  
  
    for (int i = 1; i < n; i++) {  
        passes++;  
        double key = list.get(i);  
        int j = i - 1;  
  
        while (j >= 0) {  
            comparisons++;  
            if (list.get(j) > key) {  
                list.set(j + 1, list.get(j));  
                j--;  
                shifts++;  
            } else {  
                break;  
            }  
        }  
        list.set(j + 1, key);  
    }  
    return new int[]{  
        passes,  
        shifts,  
        comparisons  
    };  
}
```

b) Time complexity

Insertion Sort has two nested loops. The outer loop runs $n-1$ times, and the inner while-loop depends on how many elements need to be shifted (GeeksforGeeks, n.d.,-b). This gives us three cases:

- Best case: $O(n)$ - if the list is already sorted, the while-loop condition is false and therefore the inner loop never executes, meaning we only do $n-1$ comparisons total (GeeksforGeeks, n.d.,-b).
- Average case: $O(n^2)$ - for a randomly ordered list, each element is on average shifted halfway through the sorted portion (GeeksforGeeks, n.d.,-b).
- Worst case: $O(n^2)$ - if the list is in reverse order, every new element must be shifted all the way to the beginning (GeeksforGeeks, n.d.,-b).

Does shuffling change the time complexity?

Randomly shuffling the list, like in the card example, does not change the asymptotic time complexity. For example, a sorted list may require fewer shifts of elements, while a shuffled list may require more. However, the overall time complexity remains the same. Big-O notation describes how the algorithm scales with input size, not the specific order of elements (GeeksforGeeks, n.d.,-b, Goodrich & Tamassia, 2006, p. 229). Shuffling only changes the initial order of the data, which may affect the actual number of operations in a specific run. However, the overall complexity class remains the same. In fact, a shuffled list represents the average case, which is still $O(n^2)$.

```
[Natural order]
Before: [9.566666666666666
After:  [8.0, 8.4, 8.
Passes: 110
Shifts: 2932
Comparisons: 3039

[Shuffled]
Before: [11.05, 12.33
After:  [8.0, 8.4, 8.
Passes: 110
Shifts: 3480
Comparisons: 3586
```

Task 3 - MergeSort

a) Implementation

Merge Sort is a divide-and-conquer algorithm (Goodrich & Tamassia, 2006, p. 678). The algorithm works in three steps: divide the array S into two sub-arrays each with $n/2$ items (Gupta, 2026, LO3). Conquer by recursively sorting each sub-array until the array is sufficiently small (size 1, which is the terminal condition where merging begins), and combine by merging the solutions to the sub-arrays into a single sorted array (Gupta, 2026, LO3).

The implementation consists of two methods. The `mergeSort()` method handles the splitting: it finds the midpoint of the list, creates a left and right sub-list, recursively calls itself on both halves, and finally calls `merge()` to combine them (Gupta, 2026, LO3). The base case is a list of size 1 or less, which is already sorted by definition (Gupta, 2026, LO3).

```
private static void merge(ArrayList<Double> dest, ArrayList<Double> left, ArrayList<Double> right) {
    mergeCount++; // Count every merge operation

    int li = 0; // pointer into left
    int ri = 0; // pointer into right
    int di = 0; // pointer into destination

    // Compare elements from both halves and place the smaller one first
    while (li < left.size() && ri < right.size()) {
        if (left.get(li) <= right.get(ri)) {
            dest.set(di, left.get(li));
            li++;
        } else {
            dest.set(di, right.get(ri));
            ri++;
        }
        di++;
    }

    // Copy any remaining elements from the left half
    while (li < left.size()) {
        dest.set(di, left.get(li));
        li++;
        di++;
    }

    // Copy any remaining elements from the right half
    while (ri < right.size()) {
        dest.set(di, right.get(ri));
        ri++;
        di++;
    }
}
```

The `merge()` method is where the actual sorting takes place. It uses three index pointers: one for the current position in the left sub-list, one for the current position in the right sub-list, and one for the current position in the destination list. At each step, the smaller of the two current elements is placed into the destination list and the corresponding pointer is advanced. This continues until one of the sub-lists is exhausted. Any remaining elements from the other sub-list are then copied directly into the destination list, since they are already sorted and guaranteed to be larger than everything already placed. The `mergeCount` counter is incremented once per call to `merge()`, allowing us to track the total number of merge operations performed.

The unique alcohol content values are extracted from the combined red and white wine dataset using a `HashSet` to avoid duplicates, giving a small list of distinct double values that are passed to the sorting algorithm.

b) Number of merge operations - does shuffling change it?

The implementation tracks every call to `merge()` using a static counter `mergeCount`. Running the algorithm on both the original and a shuffled version of the list confirms that the number of merge operations is identical in both cases, always equal to $n-1$, where n is the number of unique alcohol values.

```
[Natural order]
Before: [8.8, 9.
After:  [8.0, 8.
Merges: 110

[Shuffled]
Before: [11.75,
After:  [8.0, 8.
Merges: 110
```

This is not a coincidence. The basic operation of Merge Sort is the comparison of elements (Goodrich & Tamassia, 2006, p. 493). Each call to `merge()` combines exactly two sub-lists into one, reducing the total number of separate sub-lists by 1 (Goodrich & Tamassia, 2006, p. 680). Starting from n single-element sub-lists and finishing with 1 fully sorted list therefore always requires exactly $n-1$ merge operations, regardless of the input order.

This is also why Merge Sort has a guaranteed time complexity of $O(n \log n)$ in all cases: best, average, and worst (GeeksforGeeks, n.d.,-c). The $\log n$ factor comes from the depth of the recursion tree, since the list is halved at each level, and the n factor comes from the work done during merging at each level (Goodrich & Tamassia, 2006, p. 680). Unlike Bubble Sort, whose optimized version can exit early on a sorted list, Merge Sort always performs the same amount of work for a given input size (Gupta, 2026, LO3). Shuffling the list does not change the number of merge operations, the depth of the recursion, or the time complexity. Only the specific element comparisons made during each merge step may differ (Marculescu, n.d.).

Task 4 - QuickSort

a) Implementation

QuickSort is another divide-and-conquer technique (Goodrich & Tamassia, 2006, p. 693). We choose a pivot element from the array, then partition the remaining elements into two groups: those smaller than the pivot (left side) and those larger than the pivot (right side).

The QuickSort implementation consists of a shared partition method and four separate sort methods, one for each pivot strategy. The partition method always treats the last element as the pivot. It uses two pointers: *i* starts at low - 1 and tracks where the next element smaller than the pivot should go, while *j* scans from low to high - 1. Each time an element smaller than the pivot is found, *i* is incremented and the element is swapped into position. After the scan, the pivot is placed at *i* + 1, its correctly sorted position (GeeksforGeeks, n.d.,-d). Each pivot strategy works by moving the chosen pivot to the last position before calling the shared partition method:

```
static int partition(ArrayList<Double> arr, int low, int high) {
    double pivot = arr.get(high);
    int i = low - 1;

    for (int j = low; j <= high - 1; j++) {
        comparisons++;
        if (arr.get(j) < pivot) {
            i++;
            swap(arr, i, j);
        }
    }

    swap(arr, i + 1, high);
    return i + 1;
}

static void swap(ArrayList<Double> arr, int i, int j) { 5 usages
    double temp = arr.get(i);
    arr.set(i, arr.get(j));
    arr.set(j, temp);
}
```

a) **First element:**

The element at low is swapped to high before partitioning.

```
static void quickSortFirst(ArrayList<Double> arr, int low, int high) {
    if (low < high) {
        // Move first element to end so partition() can use it as pivot
        swap(arr, low, high);
        int pi = partition(arr, low, high);
        quickSortFirst(arr, low, high: pi - 1);
        quickSortFirst(arr, low: pi + 1, high);
    }
}
```

b) **Last element:**

No swap needed partition already uses the last element.

```
static void quickSortLast(ArrayList<Double> arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSortLast(arr, low, high: pi - 1);
        quickSortLast(arr, low: pi + 1, high);
    }
}
```

c) **Random element:**

A random index between low and high is selected and swapped to high.

```
static void quickSortRandom(ArrayList<Double> arr, int low, int high) {
    if (low < high) {
        // Move random element to end so partition() can use it as pivot
        int randomIndex = low + new Random().nextInt(bound: high - low + 1);
        swap(arr, randomIndex, high);
        int pi = partition(arr, low, high);
        quickSortRandom(arr, low, high: pi - 1);
        quickSortRandom(arr, low: pi + 1, high);
    }
}
```

d) **Median of three:**

The medianOfThree helper finds the median value among the first, middle, and last elements and returns its index, which is then swapped to high.

```

static void quickSortMedian(ArrayList<Double> arr, int low, int high) { 3 usages
    if (low < high) {
        // Find median of first, middle and last element, move it to end
        int mid = low + (high - low) / 2;
        int medianIndex = medianOfThree(arr, low, mid, high);
        swap(arr, medianIndex, high);
        int pi = partition(arr, low, high);
        quickSortMedian(arr, low, high: pi - 1);
        quickSortMedian(arr, low: pi + 1, high);
    }
}

// Returns the index of the median value among arr[a], arr[b], arr[c]
static int medianOfThree(ArrayList<Double> arr, int a, int b, int c) { 1 usage 2
    double va = arr.get(a), vb = arr.get(b), vc = arr.get(c);
    if ((va <= vb && vb <= vc) || (vc <= vb && vb <= va)) return b;
    if ((vb <= va && va <= vc) || (vc <= va && va <= vb)) return a;
    return c;
}

```

- Best case: **$O(n \log n)$** - this occurs when the pivot divides the array into two roughly equal halves each time. The array is split into $\log n$ levels, and each level does n comparisons, giving $n \times \log n$ total operations (GeeksforGeeks, n.d.,-d).
- Average case: **$O(n \log n)$** - for most random inputs, the pivot will create reasonably balanced partitions. The performance stays close to the best case (GeeksforGeeks, n.d.,-d).
- Worst case: **$O(n^2)$** - this occurs when the pivot is always the smallest or largest element, creating one empty sub-array and one with $n-1$ elements. The recursion depth becomes n instead of $\log n$. This happens for example when using the first or last element as pivot on an already sorted list (GeeksforGeeks, n.d.,-d).

This is why pivot selection strategy matters. Median-of-three and random pivot help avoid the worst case by making it unlikely that the pivot is always an extreme value (Gupta, 2026, LO3).

b) Comparisons for each pivot strategy

A static comparisons counter is incremented once per iteration inside the partition, allowing the total number of comparisons for each strategy to be tracked independently.

```
[a. First element as pivot]
After:      [8.0, 8.4, 8.5, 8.6, 8.7, 8.8, 8.9, 9.0]
Comparisons: 808

[b. Last element as pivot]
After:      [8.0, 8.4, 8.5, 8.6, 8.7, 8.8, 8.9, 9.0]
Comparisons: 640

[c. Random element as pivot]
After:      [8.0, 8.4, 8.5, 8.6, 8.7, 8.8, 8.9, 9.0]
Comparisons: 929

[d. Median of three as pivot]
After:      [8.0, 8.4, 8.5, 8.6, 8.7, 8.8, 8.9, 9.0]
Comparisons: 613
```

a) *The number of comparisons.*

This is because different pivots create differently balanced partitions (Goodrich & Tamassia, 2006, p. 505). A pivot near the median value splits the array evenly, resulting in fewer recursive calls and comparisons. A pivot near the minimum or maximum creates unbalanced partitions, leading to more comparisons (GeeksforGeeks, n.d.,-d).

b) *Which pivot selection strategy works best.*

Median-of-three performed best with only 613 comparisons. This is because it selects the median of the first, middle and last element, which is more likely to be close to the true median of the sub-array (Gupta, 2026, LO3). This creates more balanced partitions and avoids the worst case scenario that can occur with first or last element strategies (Goodrich & Tamassia, 2006, p. 701).

Conclusion

Having implemented and tested all four sorting algorithms on the unique alcohol content values from the Wine Quality Dataset, our results match what we anticipated based on course lectures.

The two iterative algorithms, Bubble Sort and Insertion Sort, both have a worst-case time complexity of $O(n^2)$ (geeksForGeeks, n.d.,-a,-b). In practice, the optimized Bubble Sort performed better than the non-optimized version due to its early-exit mechanism, and Insertion Sort outperformed both Bubble Sort variants. The two recursive algorithms, Merge Sort and QuickSort, both achieved $O(n \log n)$ time complexity (geeksForGeeks, n.d.,-c,-d), and performed significantly faster in practice.

For QuickSort specifically, the median-of-three pivot strategy required the fewest comparisons (613), followed by last element (640), first element (808), and random element (929). This demonstrates that pivot selection has a measurable impact on performance even for a small dataset.

Since the wine alcohol dataset is small (a limited number of unique values), the differences in execution time between algorithms are minimal. However, the differences in comparison counts and theoretical complexity clearly illustrate why algorithm choice matters at scale. For larger datasets, Merge Sort and QuickSort would be the appropriate choice, while Bubble Sort and Insertion Sort would become impractical with real world scalable applications.

Reflections

Working through this exam gave us a clearer understanding of how sorting algorithms behave in practice compared to what theory alone describes. One thing that stood out was how much pivot selection affected QuickSort's performance. We learned that median-of-three is on average the most reliable strategy, because selecting the median of first, middle and last element makes it unlikely that the pivot ends up being the smallest or largest value in the array. This gives more balanced partitions and fewer comparisons overall.

We also found it valuable to implement the optimized and non-optimized Bubble Sort side by side. Seeing the swapped flag eliminate unnecessary passes on a nearly sorted list.

Additionally, the random pivot strategy in QuickSort introduces variability between runs. Our comparison count for this strategy may differ slightly on repeated executions, which should be kept in mind when interpreting the results.

References

- Alake, R. (2025). Bubble sort time complexity and algorithm explained. BuiltIn.
<https://builtin.com/data-science/bubble-sort-time-complexity>
- Codecademy. (n.d.). Time complexity of bubble sort explained with examples. Codecademy.
<https://www.codecademy.com/article/time-complexity-of-bubble-sort>
- Cortez, P., Cerdeira, A., Almeida, F., Matos, T., & Reis, J. (2009). Modeling wine preferences by data mining from physicochemical properties. *Decision Support Systems*, 47(4), 547-553.
<https://doi.org/10.1016/j.dss.2009.05.016>
- GeeksforGeeks (n.d.). Bubble Sort Algorithm. GeeksforGeeks.
<https://www.geeksforgeeks.org/dsa/bubble-sort-algorithm/>
- GeeksforGeeks (n.d.). Insertion Sort Algorithm. GeeksforGeeks.
<https://www.geeksforgeeks.org/dsa/insertion-sort-algorithm/>
- GeeksforGeeks (n.d.). Merge Sort Algorithm. GeeksforGeeks.
<https://www.geeksforgeeks.org/dsa/merge-sort/>
- GeeksforGeeks (n.d.). Time complexity analysis of Bubble Sort. GeeksforGeeks.
<https://www.geeksforgeeks.org/dsa/time-and-space-complexity-analysis-of-bubble-sort/>
- GeeksforGeeks (n.d.). Quick Sort Algorithm. GeeksforGeeks.
<https://www.geeksforgeeks.org/dsa/quick-sort-algorithm/>
- Goodrich, M. T., & Tamassia, R. (2006). *Data structures and algorithms in Java* (4th ed.). John Wiley & Sons.
- Gupta, R. (2026). Sorting algorithms [Lecture slides, LO3]. PG4200 Data Structures and Algorithms, Kristiania University College.
- Interviewer live (2023). Mastering bubble sort in java. Interviewer live.
<https://interviewer.live/java/mastering-bubble-sort-in-java/>
- Marculescu, B. (n.d.). Runtime analysis and sorting [Lecture slides]. PG4200 Algorithms and Data Structures, Kristiania University College.